

Foundations of Machine Learning

Linear Regression, Logistic Regression, Support Vector Machines
and Dimensionality Reduction

Sayan CHAKI

Academic year 2026–2027

LIRIS (UMR 5205 CNRS)

École Centrale de Lyon • Université Lumière Lyon 2 • INSA Lyon

About this course

Goals

- Understand the **mathematics** behind four cornerstone methods
- Implement each method **from scratch** in NumPy
- Use **scikit-learn** correctly: pipelines, scaling, cross-validation
- Diagnose over/underfitting and choose hyperparameters

Prerequisites

- Linear algebra, calculus, basic probability
- Python, NumPy, Matplotlib

Companion Colab notebooks

1. `01_linear_regression.ipynb`
2. `02_logistic_regression.ipynb`
3. `03_svm.ipynb`
4. `04_dimensionality_reduction.ipynb`

Open in Colab: *File* → *Upload notebook*, or place them in Google Drive and open with Colaboratory. All datasets ship with scikit-learn.

Outline

Machine Learning Setup

Linear Regression

Logistic Regression

Support Vector Machines

Dimensionality Reduction

Synthesis

Machine Learning Setup

Supervised learning in one slide

- Data: $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$, drawn i.i.d. from an unknown distribution $P(\mathbf{x}, y)$
- Inputs $\mathbf{x}_i \in \mathbb{R}^d$; targets $y_i \in \mathbb{R}$ (**regression**) or $y_i \in \{1, \dots, K\}$ (**classification**)
- Hypothesis class $\mathcal{H} = \{f_\theta\}$, loss $\ell(f(\mathbf{x}), y)$

Risk and empirical risk minimisation (ERM)

$$R(f) = \mathbb{E}_{(\mathbf{x}, y) \sim P} [\ell(f(\mathbf{x}), y)] \quad \hat{R}_n(f) = \frac{1}{n} \sum_{i=1}^n \ell(f(\mathbf{x}_i), y_i) \quad \hat{f} = \arg \min_{f \in \mathcal{H}} \hat{R}_n(f) + \lambda \Omega(f)$$

- $\Omega(f)$: regularisation, controls the **capacity** of the model
- Every method in this course is an instance of this template

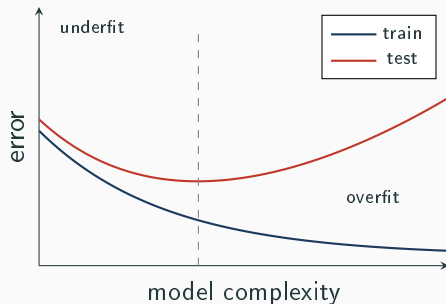
Generalisation: bias, variance and model selection

For squared loss, with $y = f^*(\mathbf{x}) + \varepsilon$,

$\text{Var}(\varepsilon) = \sigma^2$:

$$\mathbb{E}[(y - \hat{f}(\mathbf{x}))^2] = \underbrace{(f^* - \mathbb{E}\hat{f})^2}_{\text{bias}^2} + \underbrace{\text{Var}(\hat{f})}_{\text{variance}} + \underbrace{\sigma^2}_{\text{noise}}$$

- Train / validation / test split
- k -fold cross-validation for hyperparameters
- The test set is touched **once**



Preprocessing essentials

- **Standardisation:** $\tilde{x}_j = (x_j - \mu_j)/\sigma_j$. Essential for gradient descent, regularisation, SVMs and PCA
- **Fit statistics on the training set only**, then apply to validation/test. Otherwise: **data leakage**
- **Categorical features:** one-hot encoding
- **Missing values:** imputation inside the pipeline

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split, cross_val_score

X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=0)
model = make_pipeline(StandardScaler(), SomeEstimator())
scores = cross_val_score(model, X_tr, y_tr, cv=5)    # scaler refit in each fold
```

Linear Regression

The linear model

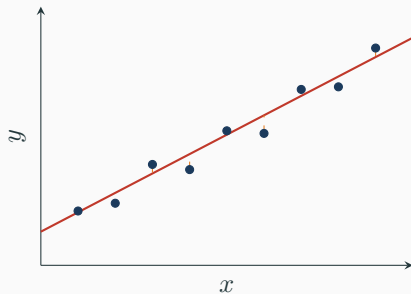
$$f_{\mathbf{w}}(\mathbf{x}) = w_0 + \sum_{j=1}^d w_j x_j = \mathbf{w}^\top \mathbf{x} \quad (\text{with } x_0 = 1)$$

Stacking n examples: design matrix $\mathbf{X} \in \mathbb{R}^{n \times (d+1)}$, targets $\mathbf{y} \in \mathbb{R}^n$.

Ordinary least squares (OLS)

$$\min_{\mathbf{w}} J(\mathbf{w}) = \frac{1}{2n} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2$$

- “Linear” means linear in the **parameters** $\mathbf{w}$, not necessarily in $\mathbf{x}$



Residuals $r_i = y_i - \mathbf{w}^\top \mathbf{x}_i$ (orange)

Closed-form solution: normal equations

Gradient of the objective:

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = -\frac{1}{n} \mathbf{X}^\top (\mathbf{y} - \mathbf{X}\mathbf{w})$$

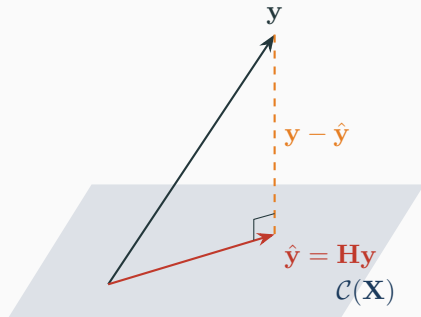
Setting it to zero gives the **normal equations**

$$\mathbf{X}^\top \mathbf{X} \mathbf{w} = \mathbf{X}^\top \mathbf{y} \quad \Longrightarrow \quad \boxed{\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}}$$

- J is convex (Hessian $\frac{1}{n} \mathbf{X}^\top \mathbf{X} \succeq 0$): any stationary point is a global minimum
- Unique iff $\mathbf{X}$ has full column rank
- In practice: never invert; use `np.linalg.lstsq` (QR / SVD). Cost $O(nd^2 + d^3)$
- If $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$, then $\hat{\mathbf{w}} = \mathbf{V}\mathbf{\Sigma}^+\mathbf{U}^\top \mathbf{y}$ (pseudo-inverse)

Geometric interpretation

- $\mathbf{X}\mathbf{w}$ ranges over the column space $\mathcal{C}(\mathbf{X})$
- OLS finds the point of $\mathcal{C}(\mathbf{X})$ closest to $\mathbf{y}$
- $\hat{\mathbf{y}} = \mathbf{X}\hat{\mathbf{w}} = \mathbf{H}\mathbf{y}$ with the **hat matrix** $\mathbf{H} = \mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top$, an orthogonal projector
- Residual $\mathbf{y} - \hat{\mathbf{y}} \perp \mathcal{C}(\mathbf{X})$, which is exactly $\mathbf{X}^\top(\mathbf{y} - \mathbf{X}\hat{\mathbf{w}}) = 0$



Probabilistic view: least squares = maximum likelihood

Assume $y_i = \mathbf{w}^\top \mathbf{x}_i + \varepsilon_i$ with $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$ i.i.d. Then

$$\log p(\mathbf{y} \mid \mathbf{X}, \mathbf{w}) = \sum_{i=1}^n \log \mathcal{N}(y_i; \mathbf{w}^\top \mathbf{x}_i, \sigma^2) = -\frac{1}{2\sigma^2} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 - \frac{n}{2} \log(2\pi\sigma^2)$$

- Maximising the likelihood in $\mathbf{w} \Leftrightarrow$ minimising squared error
- $\hat{\sigma}_{\text{MLE}}^2 = \frac{1}{n} \|\mathbf{y} - \mathbf{X}\hat{\mathbf{w}}\|^2$
- Adding a Gaussian prior $\mathbf{w} \sim \mathcal{N}(0, \tau^2 \mathbf{I})$ and taking the MAP estimate gives **ridge regression** with $\lambda = \sigma^2 / \tau^2$
- Laplace prior $\Rightarrow$ **lasso**

Gauss–Markov

Under zero-mean, homoscedastic, uncorrelated noise, OLS is the best linear unbiased estimator (BLUE).

Gradient descent

When d or n is large, iterate instead of solving:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla_{\mathbf{w}} J(\mathbf{w}^{(t)}) = \mathbf{w}^{(t)} + \frac{\eta}{n} \mathbf{X}^\top (\mathbf{y} - \mathbf{X} \mathbf{w}^{(t)})$$

- **Batch GD:** full gradient
- **SGD:** one sample per step, $O(d)$ per update
- **Mini-batch:** the deep learning default
- Converges if $\eta < 2/L$, $L = \lambda_{\max}(\frac{1}{n} \mathbf{X}^\top \mathbf{X})$
- Poor conditioning $\Rightarrow$ zig-zag. Fix:
 standardise features

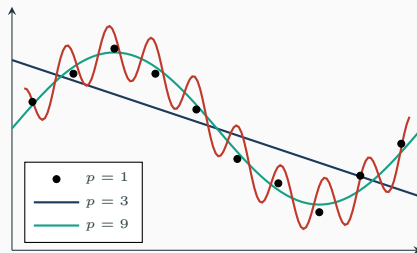
```
def gd(X, y, lr=0.1, epochs=500):  
    n, d = X.shape  
    w = np.zeros(d)  
    for _ in range(epochs):  
        grad = -X.T @ (y - X @ w) / n  
        w -= lr * grad  
    return w
```

Nonlinear features: basis expansion

Replace $\mathbf{x}$ by a feature map $\phi(\mathbf{x})$; the model stays linear in $\mathbf{w}$:

$$f(x) = \mathbf{w}^\top \phi(x), \quad \phi(x) = (1, x, x^2, \dots, x^p)$$

- Polynomial, spline, RBF, Fourier features
- Degree p controls capacity
- Too small p : **underfit**; too large: **overfit**
- PolynomialFeatures in scikit-learn



Ridge (ℓ_2)

$$\min_{\mathbf{w}} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 + \lambda \|\mathbf{w}\|_2^2$$

$$\hat{\mathbf{w}}_{\text{ridge}} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$$

- Always invertible for $\lambda > 0$
- Shrinks coefficients smoothly
- Helps with collinearity

Lasso (ℓ_1)

$$\min_{\mathbf{w}} \frac{1}{2n} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 + \lambda \|\mathbf{w}\|_1$$

- No closed form; coordinate descent with soft-thresholding
- Produces **sparse** solutions: feature selection
- **Elastic net:** $\alpha \|\mathbf{w}\|_1 + \frac{1-\alpha}{2} \|\mathbf{w}\|_2^2$
- Do not penalise the intercept; standardise first

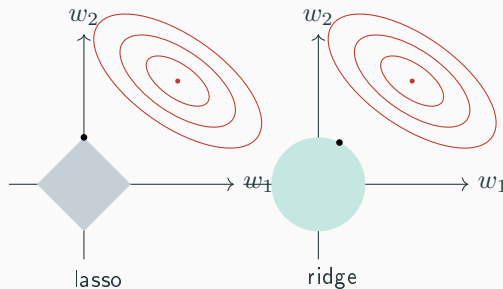
Why does ℓ_1 give sparsity?

Constrained form: $\min \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2$ s.t.
 $\|\mathbf{w}\|_q \leq t$.

- Loss level sets are ellipses around $\hat{\mathbf{w}}_{\text{OLS}}$
- The ℓ_1 ball has **corners** on the axes
- The first contact point is typically a corner, so some $w_j = 0$
- The ℓ_2 ball is round: contact generally off-axis

Soft-thresholding operator:

$$S_\lambda(z) = \text{sign}(z) \max(|z| - \lambda, 0)$$



Evaluating a regression model

- $\text{MSE} = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$, $\text{RMSE} = \sqrt{\text{MSE}}$
- $\text{MAE} = \frac{1}{n} \sum |y_i - \hat{y}_i|$ (robust to outliers)
- $R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$

Diagnostics

- Residuals vs fitted: look for structure
- Q-Q plot of residuals
- Heteroscedasticity (funnel shape)
- Influential points, leverage h_{ii}
- Learning curves

Pitfalls

Extrapolation outside the training range, correlated features making coefficients unstable, interpreting coefficients causally.

Linear regression in scikit-learn

```
from sklearn.datasets import load_diabetes
from sklearn.linear_model import LinearRegression, RidgeCV, LassoCV
from sklearn.metrics import mean_squared_error, r2_score

X, y = load_diabetes(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=0)

models = {
    "OLS": make_pipeline(StandardScaler(), LinearRegression()),
    "Ridge": make_pipeline(StandardScaler(), RidgeCV(alphas=np.logspace(-3, 3, 50))),
    "Lasso": make_pipeline(StandardScaler(), LassoCV(cv=5, random_state=0)),
}

for name, m in models.items():
    m.fit(X_tr, y_tr)
    pred = m.predict(X_te)
    print(f"{name:6s} RMSE={mean_squared_error(y_te, pred)**0.5:.2f} R2={r2_score(y_te, pred):.3f}")
```

Colab lab: 01_linear_regression.ipynb: OLS from scratch, GD, polynomial overfitting, ridge/lasso paths

Logistic Regression

From regression to classification

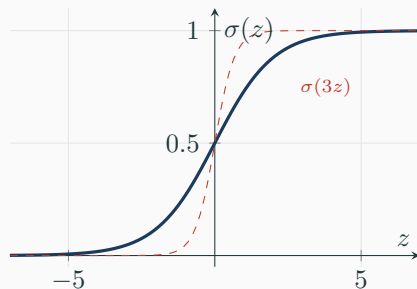
Binary labels $y \in \{0, 1\}$. Linear regression is a poor fit:

- Outputs unbounded, not probabilities
- Sensitive to points far from the boundary

Model the **posterior probability**:

$$p(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x}), \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

- $\sigma(-z) = 1 - \sigma(z)$,
 $\sigma'(z) = \sigma(z)(1 - \sigma(z))$
- **Log-odds are linear:** $\log \frac{p}{1-p} = \mathbf{w}^\top \mathbf{x}$



Maximum likelihood and cross-entropy

With $p_i = \sigma(\mathbf{w}^\top \mathbf{x}_i)$, each label is Bernoulli:

$$p(y_i \mid \mathbf{x}_i, \mathbf{w}) = p_i^{y_i} (1 - p_i)^{1-y_i}$$

The negative log-likelihood is the **binary cross-entropy (log-loss)**:

$$J(\mathbf{w}) = -\frac{1}{n} \sum_{i=1}^n \left[y_i \log p_i + (1 - y_i) \log(1 - p_i) \right]$$

Gradient and Hessian

$$\nabla J(\mathbf{w}) = \frac{1}{n} \mathbf{X}^\top (\mathbf{p} - \mathbf{y}), \quad \nabla^2 J(\mathbf{w}) = \frac{1}{n} \mathbf{X}^\top \mathbf{S} \mathbf{X}, \quad \mathbf{S} = \text{diag}(p_i(1 - p_i)) \succeq 0$$

- Same form as linear regression, with $\hat{y}_i$ replaced by p_i
- J is **convex**, but there is **no closed form**

Gradient descent

$$\mathbf{w} \leftarrow \mathbf{w} - \frac{\eta}{n} \mathbf{X}^\top (\sigma(\mathbf{X}\mathbf{w}) - \mathbf{y})$$

Newton / IRLS

$$\mathbf{w} \leftarrow \mathbf{w} - (\mathbf{X}^\top \mathbf{S} \mathbf{X})^{-1} \mathbf{X}^\top (\mathbf{p} - \mathbf{y})$$

Each step is a weighted least-squares problem (iteratively reweighted least squares), quadratic convergence.

Solvers in scikit-learn

- lbfgs: default, quasi-Newton
- liblinear: small data, ℓ_1
- saga: large data, elastic net
- newton-cg, newton-cholesky

Separable data

If classes are linearly separable, the MLE does not exist: $\|\mathbf{w}\| \rightarrow \infty$. Regularisation fixes this.

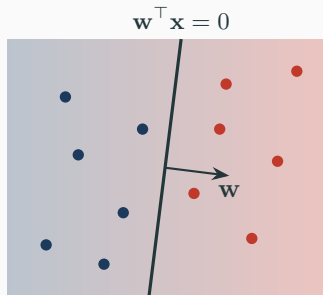
Decision boundary and regularisation

- Predict $\hat{y} = 1$ iff $p(y = 1 \mid \mathbf{x}) \geq \tau$ (default $\tau = 0.5$)
- With $\tau = 0.5$: boundary is the hyperplane $\mathbf{w}^\top \mathbf{x} = 0$
- $\|\mathbf{w}\|$ controls the **sharpness** of the transition
- Regularised objective:

$$J(\mathbf{w}) + \frac{\lambda}{2} \|\mathbf{w}\|^2$$

scikit-learn uses $C = 1/\lambda$ (large C = weak regularisation)

- Nonlinear boundaries: feature expansion, as in regression



Multiclass: softmax regression

For K classes, one weight vector per class $\mathbf{w}_1, \dots, \mathbf{w}_K$:

$$p(y = k \mid \mathbf{x}) = \text{softmax}(\mathbf{z})_k = \frac{\exp(\mathbf{w}_k^\top \mathbf{x})}{\sum_{j=1}^K \exp(\mathbf{w}_j^\top \mathbf{x})}$$

Loss (categorical cross-entropy) with one-hot targets y_{ik} :

$$J(\mathbf{W}) = -\frac{1}{n} \sum_i \sum_k y_{ik} \log p_{ik}, \quad \nabla_{\mathbf{w}_k} J = \frac{1}{n} \sum_i (p_{ik} - y_{ik}) \mathbf{x}_i$$

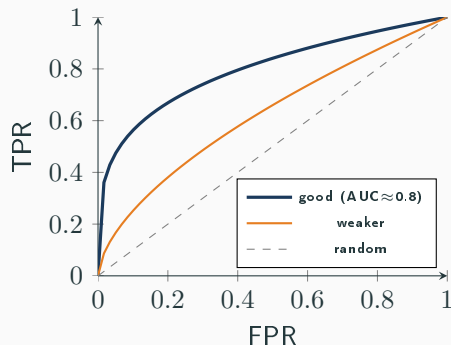
- Parameters are identifiable only up to a common shift: regularise or fix $\mathbf{w}_K = 0$
- Alternative: **one-vs-rest**, K independent binary classifiers
- Numerical stability: subtract $\max_k z_k$ before exponentiating (log-sum-exp trick)
- Softmax regression = a neural network with no hidden layer

Evaluating classifiers

Confusion matrix

	pred +	pred -
true +	TP	FN
true -	FP	TN

- Precision = $\frac{TP}{TP+FP}$
- Recall (TPR) = $\frac{TP}{TP+FN}$
- $F_1 = \frac{2PR}{P+R}$
- Accuracy misleads on imbalanced data



ROC: vary threshold τ . AUC = probability a random positive scores above a random negative. Use PR curves under strong imbalance.

Logistic regression in scikit-learn

```
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import classification_report, roc_auc_score

X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, stratify=y, test_size=0.25, random_state=0)

pipe = make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000))
grid = GridSearchCV(pipe, {"logisticregression__C": np.logspace(-3, 3, 13)},
                    cv=5, scoring="roc_auc").fit(X_tr, y_tr)

proba = grid.predict_proba(X_te)[: , 1]
print(grid.best_params_, "test AUC =", roc_auc_score(y_te, proba))
print(classification_report(y_te, grid.predict(X_te)))
```

Colab lab: 02_logistic_regression.ipynb: from-scratch GD and Newton,
boundaries, ROC/PR, softmax on digits

Support Vector Machines

Which separating hyperplane?

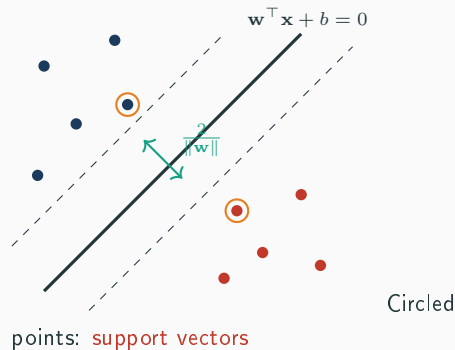
Labels $y_i \in \{-1, +1\}$, classifier

$$f(\mathbf{x}) = \text{sign}(\mathbf{w}^\top \mathbf{x} + b).$$

- Many hyperplanes separate the data
- SVM idea: choose the one with the **largest margin**
- Distance from $\mathbf{x}_i$ to the hyperplane:

$$\frac{y_i(\mathbf{w}^\top \mathbf{x}_i + b)}{\|\mathbf{w}\|}$$

- Scale so that $\min_i y_i(\mathbf{w}^\top \mathbf{x}_i + b) = 1$;
margin width $= 2/\|\mathbf{w}\|$



Hard-margin and soft-margin SVM

Hard margin (separable data)

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 \quad \forall i$$

A convex quadratic program. Real data are noisy and overlapping, so add slack variables $\xi_i \geq 0$:

Soft margin

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

- $\xi_i = 0$: correct and outside margin; $0 < \xi_i \leq 1$: inside margin; $\xi_i > 1$: misclassified
- C trades margin width against violations: large $C \Rightarrow$ low bias, high variance

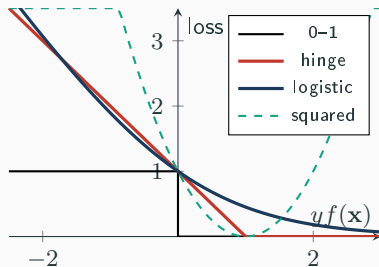
The hinge loss view

Eliminating ξ_i gives unconstrained regularised ERM:

$$\min_{\mathbf{w}, b} \frac{\lambda}{2} \|\mathbf{w}\|^2 + \frac{1}{n} \sum_i \max(0, 1 - y_i f(\mathbf{x}_i))$$

with $\lambda = 1/(nC)$.

- Hinge loss is a convex surrogate of the 0–1 loss
- Zero loss once $yf(\mathbf{x}) \geq 1$: only points near the boundary matter
- Logistic loss $\log(1 + e^{-yf})$ never reaches zero
- Enables SGD training
(`SGDClassifier(loss="hinge")`)



Lagrangian duality

Lagrangian of the soft-margin problem with multipliers $\alpha_i, \mu_i \geq 0$:

$$\mathcal{L} = \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_i \xi_i - \sum_i \alpha_i [y_i(\mathbf{w}^\top \mathbf{x}_i + b) - 1 + \xi_i] - \sum_i \mu_i \xi_i$$

Stationarity: $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$, $\sum_i \alpha_i y_i = 0$, $0 \leq \alpha_i \leq C$.

Dual problem

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j \quad \text{s.t.} \quad 0 \leq \alpha_i \leq C, \quad \sum_i \alpha_i y_i = 0$$

- The data appear **only through inner products** $\mathbf{x}_i^\top \mathbf{x}_j$
- Prediction: $f(\mathbf{x}) = \sum_i \alpha_i y_i \mathbf{x}_i^\top \mathbf{x} + b$

KKT conditions and support vectors

Complementary slackness: $\alpha_i [y_i f(\mathbf{x}_i) - 1 + \xi_i] = 0$ and $(C - \alpha_i)\xi_i = 0$.

α_i	Position of $\mathbf{x}_i$	Role
$\alpha_i = 0$	$y_i f(\mathbf{x}_i) \geq 1$, outside margin	irrelevant to $\mathbf{w}$
$0 < \alpha_i < C$	$y_i f(\mathbf{x}_i) = 1$, on the margin	free support vector
$\alpha_i = C$	$y_i f(\mathbf{x}_i) \leq 1$, inside/wrong side	bounded support vector

- The solution is **sparse** in α : removing non-SVs does not change the model
- b is recovered from any free SV: $b = y_i - \sum_j \alpha_j y_j \mathbf{x}_j^\top \mathbf{x}_i$ (average in practice)
- Leave-one-out error $\leq \#SV/n$

The kernel trick

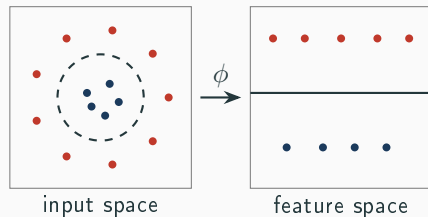
Map $\mathbf{x} \mapsto \phi(\mathbf{x})$ into a high (possibly infinite) dimensional space and replace every inner product by a **kernel**:

$$k(\mathbf{x}, \mathbf{x}') = \langle \phi(\mathbf{x}), \phi(\mathbf{x}') \rangle$$

$$f(\mathbf{x}) = \sum_i \alpha_i y_i k(\mathbf{x}_i, \mathbf{x}) + b$$

Example. In $\mathbb{R}^2$, $k(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^\top \mathbf{z})^2$ corresponds to $\phi(\mathbf{x}) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$.

Mercer: k is valid iff every Gram matrix $\mathbf{K}_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$ is positive semidefinite.



Common kernels and hyperparameters

Kernel	$k(\mathbf{x}, \mathbf{x}')$	Hyperparameters
Linear	$\mathbf{x}^\top \mathbf{x}'$	C
Polynomial	$(\gamma \mathbf{x}^\top \mathbf{x}' + r)^p$	C, γ, r, p
RBF (Gaussian)	$\exp(-\gamma \ \mathbf{x} - \mathbf{x}'\ ^2)$	C, γ
Sigmoid	$\tanh(\gamma \mathbf{x}^\top \mathbf{x}' + r)$	not always PSD

RBF intuition

- γ small: smooth boundary, can underfit
- γ large: each point is an island, overfits
- Search C, γ on a **log grid** jointly

Building kernels

- Sums, products, positive scalings of kernels are kernels
- Kernels exist for strings, graphs, sets, distributions

Solving SVMs and practical matters

Algorithms

- **SMO** (Platt): optimise two α 's at a time analytically; used by `libsvm` (SVC)
- Kernel SVM cost: between $O(n^2)$ and $O(n^3)$, memory for $\mathbf{K}$
- **Linear SVM**: `LinearSVC` (`liblinear`), coordinate descent, scales to millions
- Large n + nonlinear: random Fourier features or Nyström + linear SVM

Extensions

- Multiclass: one-vs-one (SVC) or one-vs-rest
- Probabilities: Platt scaling (`probability=True`), costly
- **SVR**: ε -insensitive loss $\max(0, |y - f(\mathbf{x})| - \varepsilon)$
- One-class SVM: novelty detection
- Class imbalance: `class_weight="balanced"`

Always scale your features before an SVM

RBF distances and the margin are meaningless if features live on different scales.

SVMs in scikit-learn

```
from sklearn.datasets import make_moons
from sklearn.svm import SVC

X, y = make_moons(n_samples=400, noise=0.25, random_state=0)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=0)

pipe = make_pipeline(StandardScaler(), SVC(kernel="rbf"))
param_grid = {"svc__C": np.logspace(-2, 3, 6), "svc__gamma": np.logspace(-3, 2, 6)}
grid = GridSearchCV(pipe, param_grid, cv=5).fit(X_tr, y_tr)

best = grid.best_estimator_
print(grid.best_params_, "test acc =", best.score(X_te, y_te))
print("support vectors per class:", best.named_steps["svc"].n_support_)
```

Colab lab: 03_svm.ipynb: hinge-loss SVM from scratch, margins, C and gamma maps, kernels, SVR

Dimensionality Reduction

Why reduce dimension?

Curse of dimensionality

- Volume grows as r^d : data become sparse
- Distances concentrate:
$$\frac{\max \|\mathbf{x} - \mathbf{x}'\| - \min \|\mathbf{x} - \mathbf{x}'\|}{\min \|\mathbf{x} - \mathbf{x}'\|} \rightarrow 0$$
- Most volume of a hypercube lies in its corners

Manifold hypothesis

- Real data (images, text) lie near a low-dimensional structure

Uses

- Visualisation (2D/3D)
- Compression, denoising
- Faster, more stable downstream models
- Removing collinearity

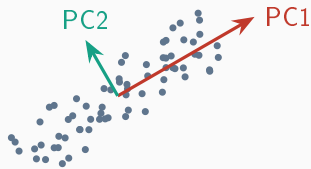
Two families

- **Feature selection**: keep a subset
- **Feature extraction**: build new features (PCA, LDA, t-SNE, UMAP, autoencoders)

Principal Component Analysis: intuition

Find orthogonal directions that capture the most **variance**.

- PC1: direction of maximal spread
- PC2: maximal spread orthogonal to PC1, and so on
- Equivalent: minimise the squared **reconstruction error** of projecting onto a k -dim subspace
- Unsupervised, linear, closed-form



PCA: derivation

Centre the data: $\mathbf{X} \leftarrow \mathbf{X} - \bar{\mathbf{x}}^\top$. Sample covariance $\Sigma = \frac{1}{n} \mathbf{X}^\top \mathbf{X}$.

Variance of the projection onto a unit vector $\mathbf{u}$:

$$\frac{1}{n} \sum_i (\mathbf{u}^\top \mathbf{x}_i)^2 = \mathbf{u}^\top \Sigma \mathbf{u}$$

Optimisation

$$\max_{\mathbf{u}} \mathbf{u}^\top \Sigma \mathbf{u} \quad \text{s.t.} \quad \|\mathbf{u}\| = 1 \quad \xrightarrow{\text{Lagrange}} \quad \Sigma \mathbf{u} = \lambda \mathbf{u}$$

- The optimum is the eigenvector with the **largest eigenvalue**; the variance captured equals λ_1
- By induction, the top- k eigenvectors $\mathbf{U}_k$ span the best k -dim subspace
- Projection $\mathbf{z} = \mathbf{U}_k^\top (\mathbf{x} - \bar{\mathbf{x}})$; reconstruction $\hat{\mathbf{x}} = \bar{\mathbf{x}} + \mathbf{U}_k \mathbf{z}$
- Reconstruction error = $\sum_{j>k} \lambda_j$ (Eckart–Young)

PCA via the SVD

Centered $\mathbf{X} = \mathbf{USV}^\top$. Then

$$\mathbf{\Sigma} = \frac{1}{n} \mathbf{VS}^2 \mathbf{V}^\top$$

- Principal directions: columns of $\mathbf{V}$
- Eigenvalues: $\lambda_j = s_j^2/n$
- Scores: $\mathbf{Z} = \mathbf{XV}_k = \mathbf{U}_k \mathbf{S}_k$
- More stable than forming $\mathbf{\Sigma}$
- Large data: randomised SVD, incremental PCA

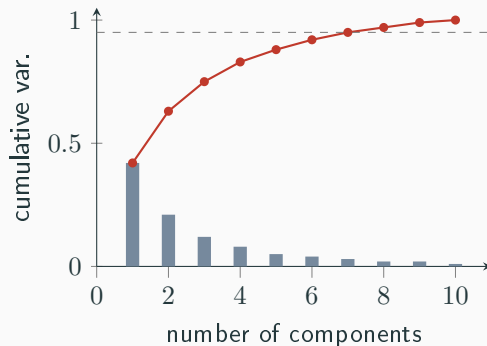
```
def pca(X, k):  
    mu = X.mean(axis=0)  
    Xc = X - mu  
    U, S, Vt = np.linalg.svd(Xc, full_matrices=  
        False)  
    comps = Vt[:k]  
    var = S**2 / (len(X) - 1)  
    ratio = var[:k] / var.sum()  
    Z = Xc @ comps.T  
    return Z, comps, ratio, mu  
  
def reconstruct(Z, comps, mu):  
    return Z @ comps + mu
```

Choosing k and practical advice

- **Explained variance ratio:**

$$\frac{\sum_{j \leq k} \lambda_j}{\sum_j \lambda_j} \geq 90-95\%$$

- **Scree plot:** look for the elbow
- **Downstream CV:** treat k as a hyperparameter
- Standardise when features have different units (PCA on the correlation matrix)
- Whitening: $\mathbf{z}_j / \sqrt{\lambda_j}$ gives unit variance
- PCA is sensitive to outliers (robust PCA exists)



Beyond linear PCA

Kernel PCA

- PCA in feature space $\phi(\mathbf{x})$ using only the Gram matrix $\mathbf{K}$
- Centre $\mathbf{K}$, then eigendecompose: $\tilde{\mathbf{K}}\alpha = n\lambda\alpha$
- Captures nonlinear structure (circles, moons)

Linear Discriminant Analysis (LDA)

- **Supervised**: maximise between-class vs within-class scatter

$$\max_{\mathbf{u}} \frac{\mathbf{u}^\top \mathbf{S}_B \mathbf{u}}{\mathbf{u}^\top \mathbf{S}_W \mathbf{u}}$$

- At most $K - 1$ components

Manifold learning / visualisation

- **t-SNE**: match neighbour probabilities (Gaussian in input, Student- t in 2D) by minimising KL divergence
 - Great clusters; global distances and cluster sizes **not** meaningful
 - Perplexity $\approx 5-50$; non-parametric
- **UMAP**: fuzzy-graph based, faster, better global structure
- Isomap, LLE, MDS
- **Autoencoders**: nonlinear PCA with neural networks

Dimensionality reduction in scikit-learn

```
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA, KernelPCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.manifold import TSNE

X, y = load_digits(return_X_y=True)          # 1797 images, 64 pixels
Xs = StandardScaler().fit_transform(X)
pca = PCA(n_components=0.95).fit(Xs)         # keep 95% of variance
print("components kept:", pca.n_components_)

Z_pca = PCA(n_components=2).fit_transform(Xs)
Z_lda = LinearDiscriminantAnalysis(n_components=2).fit_transform(Xs, y)
Z_tsne = TSNE(n_components=2, perplexity=30, random_state=0).fit_transform(Xs)
# PCA as preprocessing inside a classifier
clf = make_pipeline(StandardScaler(), PCA(n_components=30), SVC())
```

Colab lab: 04_dimensionality_reduction.ipynb: PCA from scratch,
eigen-digits, reconstructions, kernel PCA, LDA, t-SNE

Synthesis

Putting it together

	Linear reg.	Logistic reg.	SVM	PCA
Task	regression	classification	classif./regr.	unsupervised
Loss	squared	log-loss	hinge	reconstruction
Regulariser	ℓ_2 / ℓ_1	ℓ_2 / ℓ_1	ℓ_2 (margin)	rank k
Convex	yes	yes	yes	(eigenproblem)
Closed form	yes	no	no (QP)	yes (SVD)
Nonlinear via	features	features	kernels	kernel PCA
Probabilistic	Gaussian MLE	Bernoulli MLE	no (Platt)	prob. PCA
Key hyperparam.	λ , degree	C	C , kernel, γ	k

- Common thread: **regularised empirical risk minimisation** over linear functions of (possibly transformed) features
- Always: scale, pipeline, cross-validate, test once

1. Show that the ridge solution can be written $\hat{\mathbf{w}} = \mathbf{X}^\top (\mathbf{X}\mathbf{X}^\top + \lambda \mathbf{I})^{-1} \mathbf{y}$. When is this form cheaper? Connect it to the kernel trick.
2. Prove that the logistic loss is convex by showing its Hessian is PSD.
3. Derive the softmax gradient $\nabla_{\mathbf{w}_k} J = \frac{1}{n} \sum_i (p_{ik} - y_{ik}) \mathbf{x}_i$.
4. Starting from the soft-margin primal, derive the dual and the box constraint $0 \leq \alpha_i \leq C$.
5. Show that $k(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^\top \mathbf{z} + 1)^2$ is a valid kernel by writing ϕ explicitly for $d = 2$.
6. Prove that the PCA reconstruction error with k components equals $\sum_{j>k} \lambda_j$.
7. **Coding:** on `load_digits`, compare the CV accuracy of logistic regression and RBF-SVM as a function of the number of PCA components. Plot and discuss.

- C. M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006. (Ch. 3, 4, 7, 12)
- T. Hastie, R. Tibshirani, J. Friedman. *The Elements of Statistical Learning*, 2nd ed. Springer, 2009.
- K. P. Murphy. *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
- B. Schölkopf, A. J. Smola. *Learning with Kernels*. MIT Press, 2002.
- C. Cortes, V. Vapnik. Support-vector networks. *Machine Learning*, 20:273–297, 1995.
- J. Platt. Sequential minimal optimization. Microsoft Research Tech. Report, 1998.
- I. T. Jolliffe. *Principal Component Analysis*, 2nd ed. Springer, 2002.
- L. van der Maaten, G. Hinton. Visualizing data using t-SNE. *JMLR*, 9:2579–2605, 2008.
- scikit-learn user guide: https://scikit-learn.org/stable/user_guide.html

Questions?

Sayan CHAKI

LIRIS • École Centrale de Lyon • Université Lyon 2 • INSA Lyon